NON~LINEAR & ADVANCED DATA STRUCTURE

UNIT~4: DATA STRUCTURES (ETCCDS202)

Submitted by:

Aryan Solanki
Roll No: 2501730347
School of Engineering & Technology
B.Tech (AI/ML)
Section-A

Binary Search Tree (BST)

A BST has the following invariant: for any node N, every key in N's left subtree is smaller than N.key, and every key in N's right subtree is larger. Inorder traversal visits nodes in the sequence: Left → Root → Right.

Because of the BST invariant, visiting Left first means all smaller keys are printed before the current node's key, and visiting Right last means all larger keys are printed after. This produces keys in non-decreasing order — the sorted sequence. This property is fundamental to BST-based sorting (Treesort, O(n log n) average).

BST Delete — Three Cases

Case	Condition	Action
Case 1 - Leaf	No children	Simply remove the node (return None)
Case 2 — One Child	One child only	Replace node with its only child
Case 3 — Two Children	Both children exist	Find inorder successor (min of right subtree), copy its key, delete successor

Case 3 is the most complex. The inorder successor is the smallest node in the right subtree — it is the next largest value after the deleted node. Replacing the deleted node's key with the successor's key and then deleting the successor preserves the BST invariant, because: (a) the successor is larger than everything in the left subtree, and (b) smaller than everything else in the right subtree.

Time Complexity Analysis

Operation	Average Case	Worst Case (Skewed)	Reason
insert()	O(log n)	O(n)	Height of tree
search()	O(log n)	O(n)	Height of tree
delete()	O(log n)	O(n)	Height + find successor
inorder()	O(n)	O(n)	Visits every node once

Worst case O(n) occurs when a BST degenerates into a linked list — for example, inserting already-sorted data: 10, 20, 30, 40 creates a right-skewed tree with height n. Balanced BSTs (AVL, Red-Black) prevent this by maintaining height = O(log n) through rotations, guaranteeing O(log n) for all operations.

Graph Representation, BFS, and DFS

Adjacency List Representation

The graph is stored as a dictionary where each key is a node and its value is a list of (neighbour, weight) tuples. For the given directed weighted graph with 6 nodes and 9 edges: **Already-Sorted Input**

Node	Adjacency List (neighbour, weight)
A	B(2), C(4)
B	D(7), E(3)
C	E(1), F(8)
D	F(5)
E	D(2), F(6)
F	(no outgoing edges)

Adjacency list uses O(V + E) space, which is more efficient than an adjacency matrix (O(V2)) for sparse graphs. For this graph: V=6, E=9, so V2=36 vs V+E=15 — adjacency list uses less than half the space.

BFS vs DFS – Key Differences

Aspect	BFS	DFS
Strategy	Level by level	Branch by branch (deep)
Data Structure	Queue (FIFO)	Stack / Recursion
Traversal from A	A → B → C → D → E → F	A → B → D → F → E → C
Space Complexity	O(V) — queue width	O(V) — recursion depth
Time Complexity	O(V + E)	O(V + E)
Real-world use	Shortest path (unweighted)	Cycle detection, topological sort

HASH TABLE WITH SEPARATE CHAINING

What is a Collision and Why It Happens

A collision occurs when two different keys hash to the same bucket index. With hash function index = key%table_size and table_size = 5:

Key	Key % 5	Bucket	Collision?
10	10 % 5 = 0	Bucket[0]	No
15	15 % 5 = 0	Bucket[0]	Yes – collides with 10
20	20 % 5 = 0	Bucket[0]	Yes – collides with 10, 15
7	7 % 5 = 2	Bucket[2]	No
12	12 % 5 = 2	Bucket[2]	Yes – collides with 7

Collisions are inevitable — by the Pigeonhole Principle, if you insert more items than there are buckets, at least one bucket must contain multiple items. Even with fewer items, hash function properties can still cause collisions (as demonstrated above with multiples of 5).

How Separate Chaining Resolves Collisions

Separate chaining maintains a list (chain) at each bucket. When a collision occurs, the new key-value pair is simply appended to that bucket's list. Retrieval scans the list linearly. Deletion removes the matching pair from the list without affecting other pairs in the same bucket.

After inserting all 5 keys and then deleting key 15 from Bucket[0]: the chain shrinks from (10,ten) → (15,fifteen) → (20,twenty) to (10,ten) → (20,twenty), while key 10 and key 20 remain unaffected.

Time Complexity Analysis

Operation	Average Case	Worst Case	Condition for Worst Case
insert()	O(1)	O(n)	All keys hash to same bucket
get()	O(1)	O(n)	All keys hash to same bucket
delete()	O(1)	O(n)	All keys hash to same bucket

Average O(1) assumes a good hash function distributes keys uniformly. The load factor (n /table_size) determines average chain length. When load factor exceeds ~0.75, most implementations resize the table (rehash) to maintain O(1) average performance. Python's built-in dict uses this strategy with open addressing rather than chaining.